XVLA Folding 算子报告

1. 模型概述

XVLA (Cross-Embodiment Vision-Language-Action) 是用于机器人控制的 VLA 模型，使用 Soft Prompt 技术实现跨 embodiment 和跨域控制。

- **论文**: https://arxiv.org/abs/2510.10274
- **框架**: LeRobot (HuggingFace)
- **模型路径**: `/home/ubuntu/stephen/02-weight/xvla-folding/`
- **Tokenize**: facebook/bart-large

---

2. 算子架构总览

2.1 输入输出

输入	形状	说明
`observation.images.image`	(B, 3, 256, 256)	相机1图像
`observation.images.image2`	(B, 3, 256, 256)	相机2图像
`observation.images.image3`	(B, 3, 224, 224)	相机3图像
`observation.state`	(B, 8)	机器人状态
`task`	str	任务描述

输出	形状	说明
`action`	(B, 20)	机器人动作

2.2 核心组件

```
输入 → [视觉编码器] → [多模态融合] → [Soft Prompt] → [动作解码器] → 动作
         (DaViT)      (Encoder-Decoder)  (可学习)       (Flow Matching)
```

---

3. 核心算子详解

3.1 XVLAPolicy

功能: 视觉-语言-动作策略模型

组成:

组件	类型	说明
`vision_encoder`	DaViT	多尺度视觉编码
`encoder`	Transformer Encoder	图像序列编码
`decoder`	Transformer Decoder	动作解码
`soft_prompt`	Learned Embedding	任务条件
`action_head`	Linear	动作输出

3.2 视觉编码器 (DaViT)

配置: `config.json > florence_config.vision_config`

```
{
  "model_type": "davit",
  "patch_size": [7, 3, 3, 3],
  "patch_stride": [4, 2, 2, 2],
  "dim_embed": [256, 512, 1024, 2048],
  "num_heads": [8, 16, 32, 64],
  "num_groups": [8, 16, 32, 64],
  "depths": [1, 1, 9, 1],
  "window_size": 12,
  "projection_dim": 1024
}
```

DaViT 结构 (4 stage, 12 层):

Stage	Depth	Dim	Heads	算子
1	1	256	8	Patch Embed
2	1	512	16	Patch Embed
3	9	1024	32	DaViT Block
4	1	2048	64	Patch Embed

每层 DaViT Block 算子:

```
x → depthwise_conv → channel_conv → window_attention → ffn → output
```

算子	Shape	参数量
`patch_embed.proj`	(C, P^2)	$C \times P^2 \times Dim$
`attn.qkv`	(Dim, Dim×3)	$Dim^2 \times 3$
`attn.proj`	(Dim, Dim)	Dim^2

算子	Shape	参数量
`mlp.fc1`	(Dim, Dim×4)	$Dim^2 \times 4$
`mlp.fc2`	(Dim×4, Dim)	$Dim^2 \times 4$
`norm1`	(Dim)	Dim
`norm2`	(Dim)	Dim

3.3 文本编码器 (Florence2 Encoder)

配置: `config.json > florence_config.text_config`

```
{
  "vocab_size": 51289,
  "d_model": 1024,
  "encoder_layers": 12,
  "decoder_layers": 12,
  "decoder_attention_heads": 16,
  "encoder_attention_heads": 16,
  "max_position_embeddings": 4096,
  "activation_function": "gelu"
}
```

Encoder 算子 (12层):

算子	Shape	参数量公式
`self_attn.q_proj`	(1024, 1024)	1024^2
`self_attn.k_proj`	(1024, 1024)	1024^2
`self_attn.v_proj`	(1024, 1024)	1024^2
`self_attn.out_proj`	(1024, 1024)	1024^2
`fc1`	(1024, 4096)	1024×4096
`fc2`	(4096, 1024)	4096×1024
`norm1`	(1024)	1024
`norm2`	(1024)	1024

Decoder 算子 (12层):

算子	Shape	参数量公式
`self_attn`	同 Encoder	同 Encoder
`cross_attn.q_proj`	(1024, 1024)	1024^2
`cross_attn.k_proj`	(1024, 1024)	1024^2

算子	Shape	参数量公式
`cross_attn.v_proj`	(1024, 1024)	1024^2
`cross_attn.out_proj`	(1024, 1024)	1024^2
`fc1`, `fc2`	同 Encoder	同 Encoder
`norm1`, `norm2`, `norm3`	同 Encoder	同 Encoder

3.4 Soft Prompt 算子

配置: `config.json`

```
{
  "num_domains": 30,
  "len_soft_prompts": 32,
  "dim_time": 32
}
```

算子	Shape	参数量公式
`soft_prompt`	(30, 32, 1024)	30×32×1024

Soft Prompt 公式:

```
prompt = domain_embedding + time_embedding
output = concat(visual_features, prompt, task_embedding)
```

3.5 动作头 (Action Head)

配置: `config.json`

```
{
  "chunk_size": 30,
  "n_action_steps": 30,
  "num_denoising_steps": 10,
  "action_mode": "ee6d",
  "max_state_dim": 20
}
```

算子	输入	输出	说明
`action_in_proj`	(B, N, 8)	(B, N, D)	状态投影
`time_emb`	(B, N)	(B, N, D)	时间嵌入
`action_out_proj`	(B, N, D)	(B, N, 20)	动作输出

3.6 图像预处理算子

代码来源: `policy_preprocessor.json`

步骤	算子	说明
1	`rename_observations_processor`	重命名观察
2	`to_batch_processor`	转为 batch
3	`tokenizer_processor`	文本 tokenize
4	`device_processor`	移至设备
5	`normalizer_processor`	归一化

归一化映射:

特征类型	归一化方式
VISUAL	IDENTITY
STATE	IDENTITY
ACTION	MEAN_STD

3.7 Flow Matching 动作生成

公式:

```
x_t = (1 - t) * noise + t * actions
v_t = model(x_t, t)
loss = || noise - v_t ||²
```

推理采样 (10 步):

```
x_0 = noise
for i in range(10):
    t = 1 - i/10
    v = model(x_t, t)
    x_{t+1} = x_t + (1/10) * v
return x_last
```

---

4. 推理流程

4.1 输入处理

```
1. 图像归一化
   image ∈ [0, 255] → (image / 255.0) × 2 - 1
   图像 resize 到 (256, 256)

2. 状态归一化
   state → (state - mean) / std

3. 任务 tokenize
   text → tokenizer → input_ids
```

4.2 前向传播

```
1. 视觉编码
   images → DaViT → visual_features
   visual_features → encoder → encoded_visual

2. 文本编码
   input_ids → encoder → text_features

3. 特征融合
   concat(encoded_visual, soft_prompt, text_features) → fused

4. 动作解码
   fused → decoder → action_features
   action_features → action_head → action

5. 后处理
   action → unnormalize → output_action
```

---

5. 参数量统计

5.1 各组件参数量

组件	参数量	占比
DaViT Vision	~200M	~40%
Florence Encoder	~120M	~24%
Florence Decoder	~120M	~24%
Soft Prompt	~40K	~0.01%
Action Head	~50K	~0.01%
总计	**~500M**	100%

5.2 与 π0.5 对比

特性	XVLA	π0.5
视觉编码	DaViT (多尺度)	SigLIP (单尺度)
语言模型	Florence2 (Seq2Seq)	Gemma (LM)
训练目标	Flow Matching	Flow Matching
Soft Prompt	30 domains	无
Action Mode	EE6D (6D + Gripper)	7DOF
动作步数	30	50

---

6. 完整模型结构

```
XVLAPolicy (~500M)
  vision_encoder: DaViTModel (~200M)
    patch_embed: Conv3d (3×7×7 → 256)
    stages: ModuleList (4 stages)
      stage0: [PatchEmbed + DaViTBlock] × 1
      stage1: [PatchEmbed + DaViTBlock] × 1
      stage2: [PatchEmbed + DaViTBlock] × 9
      stage3: [PatchEmbed + DaViTBlock] × 1
    projector: Linear (2048 → 1024)
  encoder: FlorenceEncoder (~120M)
    embed_tokens: Embedding (51289 × 1024)
    layers: ModuleList (12层)
      LayerNorm
      SelfAttention (Multi-Head)
      LayerNorm
      FeedForward (GELU)
    norm: LayerNorm
  decoder: FlorenceDecoder (~120M)
    embed_tokens: Embedding (51289 × 1024)
    layers: ModuleList (12层)
      LayerNorm
      SelfAttention
      CrossAttention
      LayerNorm
      FeedForward
    norm: LayerNorm
  soft_prompt: Embedding (30 × 32 × 1024)
  action_head: ActionHead
    state_encoder: Linear (8 → 1024)
    time_encoder: SinusoidalPosEmb (32)
    out_proj: Linear (1024 → 20)
```

---

7. 关键算子公式

算子	公式	PyTorch
DaViT Attention	Depthwise + Channel + Window	手动实现
Cross Attention	`softmax(QK^T/√d)V`	`F.scaled_dot_product_attention`
Flow Matching	`x_{t+1} = x_t + dt·v_t`	手动实现
EE6D Action	(x, y, z, roll, pitch, yaw, gripper)	7 + 13

---

8. 参考资料

- 论文: https://arxiv.org/abs/2510.10274
- LeRobot: https://github.com/huggingface/lerobot
- X-VLA: https://github.com/2toinf/X-VLA
- 模型: https://huggingface.co/lerobot/xvla-folding